SCHOOL ADMISSION SYSTEM

Architecture, System Design & Delivery Blueprint

A minimal, assignment-aligned implementation plan for first-come-first-served admission, temporary seat holds and simulated payment confirmation.

Prepared for	Assignment deadline	Required stack
Sudipta Roy	11 November 2026, 11:59:59 PM (Bangladesh time, UTC+6)	.NET 8+ / PostgreSQL / React 18+ / Docker

DESIGN TARGET

Keep the codebase small and explainable. Put seat correctness in PostgreSQL transactions, keep the API contract exact, and build only the parent/admin screens needed to complete the assignment.

Version 2.1 | 8 November 2026

Status: architecture and implementation guidance. This document is not a compiled application and does not claim that software is bug-free.

Read this first

RECOMMENDED IMPLEMENTATION

One ASP.NET Core API + one React SPA + one PostgreSQL database. Docker Compose starts the database, a one-shot migration step, the API and the frontend. An optional second API container demonstrates the multi-process bonus.

This document combines the strongest parts of the two supplied design drafts, corrects the identified problems, and reduces unnecessary infrastructure. It is grounded in the five-page assignment PDF [S1]. Where the PDF is silent, the choice is labelled as an assumption rather than presented as a requirement.

Non-negotiable acceptance points

- A class-and-shift combination is one independent admission pool with its own totalSeats.
- No lottery: first successful seat hold wins. At every moment, active HELD plus CONFIRMED must not exceed totalSeats.
- HOLD_DURATION_SECONDS controls the hold duration; default is 600 seconds. Expiry frees capacity without requiring a request.
- One student may have at most one active HELD or CONFIRMED application per class/shift.
- Only the same student may confirm their own unexpired HELD application. Repeated confirmation must not increase occupancy twice.
- All required parent and admin actions are available from the React UI. No Postman or direct database action is needed for normal use.
- The UI shows EXPIRED without a manual reload, disables fee confirmation, refreshes class counts periodically, and restores a student's applications after reopening the browser.
- PostgreSQL data survives container restart. `docker compose up --build` starts the complete default application.
- The required endpoints and response field names remain intact. Backend tests include at least one real concurrency test.
- Public GitHub repository includes commit history, backend/frontend Dockerfiles, root docker-compose.yml, README.md, ASSUMPTIONS.md and AI_USAGE.md when AI is used.

How to use this blueprint

Part	What it gives you
1-3	Requirements, scope and small architecture decisions.
4-6	Domain model, database, concurrency and exact API contract.
7-9	Responsive UI, code structure and Docker/environment design.
10-12	Automated tests, delivery plan and final acceptance checklist.
Appendices	Copy-ready ASSUMPTIONS.md and AI_USAGE.md content.

Source convention: "Required" means stated in the assignment [S1]. "Assumption" means a practical interpretation that should be recorded in ASSUMPTIONS.md.

1. Requirements and scope

1.1 Functional requirements

Actor	Action	Expected result
School admin	Create a class/shift with a name and seat count.	A new independent seat pool is created; its initial availableSeats equals totalSeats.
Parent	Enter the student's 17-digit birth-registration number.	The value is used as the student identifier in X-Student-Id. No login screen is required by the brief.
Parent	Browse classes/shifts and apply.	The API either creates a temporary HELD application (201) or returns a meaningful conflict (409).

Actor	Action	Expected result
Parent	View own applications.	GET /me/holds shows current and historical applications for that student.
Parent	Confirm admission fee before expiry.	The demo assumes payment succeeded and changes HELD to CONFIRMED. No real gateway is integrated.
Parent	Cancel an unconfirmed application.	DELETE changes an unexpired HELD application to CANCELLED and releases the seat.
System	Expire old holds.	Expired holds stop consuming capacity without any API call; a background worker records EXPIRED.

1.2 Required UI surfaces

- Parent area: student-ID field, class/shift list, held/confirmed/available counts, Apply action, My Applications, status and expiry countdown, simulated fee-confirmation action, cancellation for a HELD application.
- Admin area: create class/shift form with name and positive totalSeats; the created pool appears in the same class list.
- All error, loading, empty, success and unavailable-service states must be visible in plain language. A blank page or console-only error is not acceptable.

1.3 Explicitly out of scope

- Real payment gateway, payment webhooks, refunds or financial records.
- Parent accounts, OTP, passwords, or identity-provider authentication.
- Lottery, waitlist, seat transfer, document upload, student profile or class-edit workflow.
- Microservices, Redis, message broker, Kubernetes, server-side rendering or offline write queue.
- Production-grade school admin roles, audit approval workflows and legal identity verification.

DEMO SECURITY BOUNDARY

The assignment deliberately has no login. X-Student-Id is a caller-supplied identifier, not authentication. The admin screen is open in this take-home version. Clearly disclose this limitation; do not present the demo as production-ready for real student records.

2. Technology and key decisions

Layer	Choice	Reason / constraint
Backend	ASP.NET Core Minimal API on .NET 10 LTS	Meets .NET 8+ requirement. .NET 8 support ends 10 Nov 2026; .NET 10 LTS is supported through 14 Nov 2028 [R1].
Data access	EF Core + Npgsql	Migrations and ordinary CRUD via EF; a few parameterized SQL statements for row locks and conditional state transitions.
Database	PostgreSQL 16 (Docker image `postgres:16-alpine`)	Transactions, row locks, partial unique indexes and durable named volume.
Frontend	React 19.x + TypeScript + Vite	Meets React 18+ requirement. React docs list 19.3 as the latest documented release on the design date [R2].
UI styling	Small custom CSS; no component framework	Responsive, accessible and easier to debug. Visual complexity is not required.
Data fetching	Small typed fetch client + React hooks	Polling every few seconds; avoid an unnecessary global state framework.

Layer	Choice	Reason / constraint
Tests	xUnit backend; Vitest + Testing Library frontend	Real PostgreSQL integration tests are required for lock/index behaviour.
Runtime	Docker Compose + Nginx static frontend	One command starts the default app. Frontend API URL is injected at container start.

2.1 Decisions that keep the build minimal

- Use one modular monolith API. Organize by feature, not by microservice.
- One database table represents one class/shift pool. Do not create separate Shift, Seat, Payment or Student tables for the take-home scope.
- Do not store mutable heldSeats/confirmedSeats counters. Derive them from holds so counter drift cannot occur.
- Only hold creation needs the class-row mutex. Confirm preserves occupied capacity; cancel and expiry reduce it.
- Use database time for every expiry decision. Browser countdown is presentation only.
- No class-name unique constraint: duplicate names are not prohibited by the brief and should not create avoidable test failures.
- Use 128-bit random hold IDs, not 64-bit IDs, because GET /holds/{holdId} has no student header in the published contract.
- Default API port is 8080. The optional second-process bonus uses a separate process/container on port 8081; do not make one process listen on two ports and call that the bonus.

2.2 Small but important assumptions

Question left open by brief	Chosen assumption
Can the same student apply to another class/shift?	Yes. The one-active-application rule is per student per class/shift, not global.
Can a student apply again after expiry or cancellation?	Yes. EXPIRED and CANCELLED are terminal history records; a new HELD application is allowed.
What if a confirmed hold is cancelled?	Reject with 409 HOLD_ALREADY_CONFIRMED. Refund/reversal is outside scope.
Can class names repeat?	Yes. IDs identify pools; duplicate names are allowed.
What proves parent/student identity?	Nothing beyond the supplied 17-digit value. This is a deliberate demo limitation.
What does “Pay Admission Fee” do?	It simulates a successful payment and confirms the hold. No money is charged.
How is an ID remembered after browser close?	Store only the active student ID in localStorage for this demo; provide a clear “Forget student ID” action and document the PII risk.

3. Architecture and runtime flow

3.1 Default runtime topology

```
Parent / Admin browser
  http://localhost:3000
        |
        | React SPA; API_BASE_URL injected at container start
        v
ASP.NET Core API  (host port 8080)
  endpoints -> feature services -> EF Core / parameterized SQL
        |
        +---- PostgreSQL 16 (named Docker volume)
        |
        +---- BackgroundService (expiry cleanup)

One-shot migration container runs before API startup.
```

Browser traffic must use a host-reachable API address such as http://localhost:8080. Never put the Docker-only hostname http://api:8080 into browser runtime configuration; the browser runs on the host, outside the Compose network.

3.2 Optional multi-process bonus topology

```
Browser / concurrency runner
     |                        |
     v                        v
API process A              API process B
host :8080                 host :8081
     |                        |
     +------------+-----------+
                  v
          Same PostgreSQL DB
          Same schema + rows
```

Compose exposes API process B only through the optional `bonus` profile. Both processes use the exact same database connection and application image. PostgreSQL row locks and the unique index coordinate them; no C# static lock or in-memory cache is involved.

3.3 Request responsibility

Component	Owns	Must not own
React UI	Input feedback, countdown display, polling, navigation between Parent/Admin areas.	Seat allocation truth, expiry authority, payment processing.
API endpoints	HTTP binding, validation, mapping results/errors to status codes.	Duplicated business rules or ad-hoc SQL in endpoint handlers.
Admission service	Transactions, capacity check, ownership/state checks, status transitions.	Presentation-specific logic.
PostgreSQL	Durable state, row-lock coordination, uniqueness and authoritative time.	UI labels or browser state.
Expiry worker	Batch-flip stale HELD rows to EXPIRED.	Correctness-critical seat release; capacity predicates already do that.

3.4 Dependency rule

Endpoint -> feature service -> DbContext / SQL. React page -> feature hook -> typed API client. Keep each layer small. Do not add a generic repository abstraction; EF Core is already the data-access boundary.

4. Domain model and state machine

4.1 Core records

Record	Fields	Meaning
Class	id, name, totalSeats, createdAt	One grade + shift. It is the seat pool and the mutex row for hold creation.
Hold	id, classId, studentId, status, createdAt, expiresAt, confirmedAt, cancelledAt	One student's application to one pool. Historical terminal rows are retained.

4.2 Status transitions

```
                +------------------+
                |       HELD       |
                +------------------+
                  |       |      |
          fee confirm|       |      |cancel by owner
          before     |       |      |before expiry
          expiry     |       |      |
                     v       v      v
               CONFIRMED  EXPIRED  CANCELLED
```

Status	Consumes capacity?	Can transition to
HELD and unexpired	Yes	CONFIRMED, EXPIRED, CANCELLED
HELD but expired by time	No	EXPIRED (stored cleanup); never CONFIRMED
CONFIRMED	Yes	Terminal in this assignment
EXPIRED	No	Terminal
CANCELLED	No	Terminal

4.3 Invariants (must always hold)

- I1 - Capacity: for each class, CONFIRMED count + unexpired HELD count <= totalSeats.
- I2 - Uniqueness: at most one HELD-or-CONFIRMED row for each (classId, studentId).
- I3 - Ownership: confirmation and cancellation require exact studentId match.
- I4 - Expiry: confirmation is permitted only while status=HELD and expiresAt is later than database time.
- I5 - Idempotency: repeating confirmation of the same owner's already-CONFIRMED hold returns success without adding another row or count.
- I6 - Durability: a committed class/hold survives API restart and ordinary `docker compose down` / `up`.

4.4 Seat accounting

```
heldSeats      = count(status = HELD AND expiresAt > databaseTime)
confirmedSeats = count(status = CONFIRMED)
availableSeats = totalSeats - heldSeats - confirmedSeats
```

Do not clamp availableSeats to zero to hide a bug. If it ever becomes negative, emit a high-severity log and fail the relevant consistency test. The transaction design must make that state impossible.

5. PostgreSQL data design

5.1 Recommended schema

```
CREATE TABLE classes (
  id          text PRIMARY KEY,
  name        varchar(100) NOT NULL CHECK (char_length(btrim(name)) BETWEEN 1 AND 100),
  total_seats integer NOT NULL CHECK (total_seats > 0),
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp()
);

CREATE TABLE holds (
  id           text PRIMARY KEY,
  class_id     text NOT NULL REFERENCES classes(id),
  student_id   varchar(17) NOT NULL CHECK (student_id ~ '^[0-9]{17}$'),
  status       varchar(12) NOT NULL CHECK
               (status IN ('HELD','CONFIRMED','EXPIRED','CANCELLED')),
  created_at   timestamptz NOT NULL,
  expires_at   timestamptz NOT NULL,
  confirmed_at timestamptz NULL,
  cancelled_at timestamptz NULL
```

```
);

-- Enforce one active application/admission per student per class/shift.
CREATE UNIQUE INDEX ux_holds_active_student_class
  ON holds (class_id, student_id)
  WHERE status IN ('HELD','CONFIRMED');

CREATE INDEX ix_holds_student_created
  ON holds (student_id, created_at DESC);
CREATE INDEX ix_holds_class_status_expiry
  ON holds (class_id, status, expires_at);
CREATE INDEX ix_holds_due_expiry
  ON holds (expires_at) WHERE status = 'HELD';
```

5.2 Schema notes

- Store birth-registration identifiers as varchar(17), never as an integer; a leading zero must be preserved.
- Use `timestamptz` and UTC instants. In .NET, use DateTime values with Kind=Utc or DateTimeOffset with offset zero consistently.
- IDs use a prefix plus 32 lowercase hexadecimal characters generated from 16 cryptographically random bytes (128 bits). Example: c_4f... (32 hex chars) and h_91... .
- Do not add a unique constraint on class name; the brief requires an ID, not a unique label.
- The partial unique index includes stale HELD rows until they are changed to EXPIRED. Before a student reapplies, expire that student's stale HELD row inside the class-locked transaction.

5.3 Counts in one database query

```
WITH db_time AS (SELECT statement_timestamp() AS ts)
SELECT c.id, c.name, c.total_seats,
  COUNT(h.id) FILTER (
    WHERE h.status = 'HELD' AND h.expires_at > db_time.ts
  ) AS held_seats,
  COUNT(h.id) FILTER (
    WHERE h.status = 'CONFIRMED'
  ) AS confirmed_seats,
  c.total_seats
    - COUNT(h.id) FILTER (
        WHERE h.status = 'HELD' AND h.expires_at > db_time.ts
      )
    - COUNT(h.id) FILTER (WHERE h.status = 'CONFIRMED') AS available_seats
FROM classes c
CROSS JOIN db_time
LEFT JOIN holds h ON h.class_id = c.id
GROUP BY c.id, c.name, c.total_seats, db_time.ts
ORDER BY c.created_at, c.id;
```

Use the same aggregate projection for GET /classes/{classId}. One query avoids N+1 counts and uses one statement-time value consistently across the result.

6. Concurrency-safe seat allocation

CORE RULE

Only creating a new HELD row can increase occupied capacity. Create-hold is serialized by locking the class row. Confirm preserves occupied capacity; cancel and expiry only decrease it.

6.1 Why this locking strategy

Approach	Decision
C# lock / SemaphoreSlim	Reject. It coordinates only one process and cannot protect two containers.
Redis / external lock	Reject. It adds infrastructure not required by the brief.
SERIALIZABLE everywhere	Reject for this small task. It adds serialization failures and retry complexity.

Approach	Decision
Mutable seat counter	Reject. Expiry/cancellation races can leak or double-release capacity.
Per-class PostgreSQL row lock for create-hold	Choose. PostgreSQL serializes create requests for the same pool across processes.

6.2 Create hold transaction

1. Begin a short READ COMMITTED transaction.
2. Lock the requested class row with SELECT id FROM classes WHERE id=@id FOR UPDATE. If no row is returned, respond 404 CLASS_NOT_FOUND.
3. After the lock is acquired, read `clock_timestamp()` from PostgreSQL. Do not reuse an application clock or transaction-start timestamp that may be stale after waiting.
4. Change this same student's stale HELD row for this class to EXPIRED. This frees the partial unique index entry so a reapplication is possible.
5. Check for an existing active/confirmed application for this student and class. Return ALREADY_APPLIED or ALREADY_ADMITTED as a 409 conflict.
6. Count CONFIRMED plus HELD with expires_at > captured database time. If count >= total_seats, return 409 NO_SEATS_AVAILABLE.
7. Insert a new HELD row with expires_at = databaseTime + HOLD_DURATION_SECONDS. Commit, then return 201.

```
await using var tx = await db.Database.BeginTransactionAsync(
    IsolationLevel.ReadCommitted, ct);

// Use the active EF transaction and a parameterized DbCommand so FOR UPDATE
// is executed directly (not composed into an outer SELECT).
var connection = db.Database.GetDbConnection();
await using var lockCommand = connection.CreateCommand();
lockCommand.Transaction = db.Database.CurrentTransaction!.GetDbTransaction();
lockCommand.CommandText = "SELECT id FROM classes WHERE id = @id FOR UPDATE";
var idParameter = lockCommand.CreateParameter();
idParameter.ParameterName = "id";
idParameter.Value = classId;
lockCommand.Parameters.Add(idParameter);
var lockedId = await lockCommand.ExecuteScalarAsync(ct);
if (lockedId is null) throw ClassNotFound();

// Read time only after acquiring the class lock, using the same connection.
await using var clockCommand = connection.CreateCommand();
clockCommand.Transaction = db.Database.CurrentTransaction!.GetDbTransaction();
clockCommand.CommandText = "SELECT clock_timestamp()";
var dbNow = (DateTime)(await clockCommand.ExecuteScalarAsync(ct))!;

await ExpireThisStudentsStaleHoldAsync(db, classId, studentId, dbNow, ct);
await EnsureStudentHasNoActiveApplicationAsync(db, classId, studentId, ct);
await EnsureCapacityAvailableAsync(db, classId, dbNow, ct);

var hold = Hold.Create(NewSecureId("h_"), classId, studentId,
                       dbNow, dbNow.AddSeconds(holdDurationSeconds));
db.Holds.Add(hold);
await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

Implementation note: import Microsoft.EntityFrameworkCore.Storage for GetDbTransaction(). The snippet shows the order, not a complete copy-paste service. Use parameterized SQL, not string concatenation. Catch PostgreSQL unique violation 23505 only when the violated constraint is `ux_holds_active_student_class`; map that specific race to the correct conflict.

6.3 Confirm: atomic conditional update

```
UPDATE holds
SET status = 'CONFIRMED', confirmed_at = clock_timestamp()
WHERE id = @hold_id
  AND student_id = @student_id
  AND status = 'HELD'
  AND expires_at > clock_timestamp();
```

If one row is updated, return 200 CONFIRMED. If zero rows are updated, load the hold and classify the result: missing -> 404; different student -> 403; already CONFIRMED for the same student -> idempotent 200; EXPIRED or time-expired HELD -> 409 HOLD_EXPIRED; CANCELLED -> 409 HOLD_CANCELLED. Do not increment counters: none exist.

Why there is no class lock: HELD -> CONFIRMED keeps the number of occupied seats unchanged. A concurrent create counts either the HELD row or the CONFIRMED row, so capacity does not increase.

6.4 Cancel: conditional update

```
UPDATE holds
SET status = 'CANCELLED', cancelled_at = clock_timestamp()
WHERE id = @hold_id
  AND student_id = @student_id
  AND status = 'HELD'
  AND expires_at > clock_timestamp();
```

One row updated -> 204. If already CANCELLED by the same student -> also 204 (idempotent). Missing -> 404; different student -> 403; EXPIRED/time-expired -> 409 HOLD_EXPIRED; CONFIRMED -> 409 HOLD_ALREADY_CONFIRMED. Cancelling only reduces occupied capacity, so no class lock is required.

6.5 Expiry without a request

Correctness comes from the time predicate in every capacity query: once expires_at <= database time, the hold stops consuming a seat even if its stored status is still HELD. A small BackgroundService updates stale rows for clean state and indexes.

```
WITH due AS (
  SELECT id FROM holds
  WHERE status = 'HELD' AND expires_at <= statement_timestamp()
  ORDER BY expires_at
  LIMIT 500
  FOR UPDATE SKIP LOCKED
)
UPDATE holds h
SET status = 'EXPIRED'
FROM due
WHERE h.id = due.id
  AND h.status = 'HELD'
  AND h.expires_at <= statement_timestamp();
```

Run this sweep every 3-5 seconds with a bounded batch. Multiple API instances may run the worker; SKIP LOCKED prevents them from waiting on or processing the same locked rows. The sweeper is housekeeping, not the mechanism that makes a seat available.

6.6 Why R1 is preserved

- Only create-hold increases active occupancy. All creates for one class must acquire the same PostgreSQL class-row lock before counting and inserting.
- Under READ COMMITTED, the capacity count is a new SQL statement after the lock is acquired, so it sees prior committed creates for that class.
- Confirm preserves occupancy; cancel and expiry only reduce it. Therefore these operations do not need the class-row lock.
- The partial unique index separately enforces one active/confirmed application per student/class, even if application code has a defect.
- Every writer of HELD rows must use the class-lock transaction. No controller, script or worker may insert a hold directly.

IMPORTANT CLOCK QUALIFICATION

PostgreSQL is the authoritative clock for the app, but `clock_timestamp()` is a wall-clock reading, not a mathematically monotonic clock. Keep database host time synchronized. Never claim that manual system-clock changes are impossible.

6.7 Multi-process bonus proof

API A and API B have independent memory and separate listening ports but share one PostgreSQL instance. PostgreSQL row-level locks and the unique index are shared database mechanisms; therefore the same create transaction serializes across

both processes. Prove this by running simultaneous requests split between ports 8080 and 8081, then asserting heldSeats + confirmedSeats <= totalSeats.

7. HTTP API contract

The following eight routes preserve the method, path, header and response fields shown in the assignment [S1]. JSON uses camelCase. Do not rename or remove the example fields. Additional non-breaking fields are unnecessary for the first version.

#	Method + path	Header / body	Success
1	POST /classes	JSON: { name, totalSeats }	201 { id, name, totalSeats }
2	GET /classes	None	200 array: id, name, totalSeats, heldSeats, confirmedSeats, availableSeats
3	GET /classes/{classId}	None	200 one class with the same seat fields
4	POST /classes/{classId}/holds	X-Student-Id	201 { holdId, classId, studentId, status, expiresAt }
5	GET /holds/{holdId}	None	200 { holdId, classId, status, expiresAt }
6	POST /holds/{holdId}/confirm	X-Student-Id	200 { holdId, status: CONFIRMED }
7	DELETE /holds/{holdId}	X-Student-Id	204 empty body
8	GET /me/holds	X-Student-Id	200 array: holdId, classId, className, status, expiresAt

7.1 Example requests and responses

```
POST /classes
Content-Type: application/json

{ "name": "Class 6 - Morning Shift", "totalSeats": 120 }

201 Created
{ "id": "c_<32-hex-chars>",
  "name": "Class 6 - Morning Shift", "totalSeats": 120 }

POST /classes/c_<id>/holds
X-Student-Id: 20152691234567890

201 Created
{ "holdId": "h_<32-hex-chars>", "classId": "c_<id>",
  "studentId": "20152691234567890", "status": "HELD",
  "expiresAt": "2026-11-08T16:00:00.000Z" }

POST /holds/h_<id>/confirm
X-Student-Id: 20152691234567890

200 OK
{ "holdId": "h_<id>", "status": "CONFIRMED" }

DELETE /holds/h_<id>
X-Student-Id: 20152691234567890

204 No Content
```

The timestamp above is illustrative only. Generate expiresAt from PostgreSQL time and serialize it as a UTC ISO-8601 string ending in Z.

7.2 Status and error mapping

HTTP	Error codes / condition
400	INVALID_REQUEST, MISSING_STUDENT_ID, INVALID_STUDENT_ID

HTTP	Error codes / condition
403	NOT_HOLD_OWNER
404	CLASS_NOT_FOUND, HOLD_NOT_FOUND, ROUTE_NOT_FOUND
409	NO_SEATS_AVAILABLE, ALREADY_APPLIED, ALREADY_ADMITTED, HOLD_EXPIRED, HOLD_CANCELLED, HOLD_ALREADY_CONFIRMED
500	INTERNAL_ERROR (generic message + traceId; no SQL or stack trace)
503	SERVICE_UNAVAILABLE when database/service is unavailable

7.3 Standard error envelope

```
{
  "error": {
    "code": "NO_SEATS_AVAILABLE",
    "message": "No seats are available in this class and shift right now.",
    "details": []
  }
}
```

7.4 Validation rules

- X-Student-Id is required on routes 4, 6, 7 and 8. Trim outer whitespace, then require exactly 17 ASCII digits: ^[0-9]{17}$. Do not parse it as a number.
- GET /holds/{holdId} has no X-Student-Id requirement in the published contract. Use a 128-bit unguessable hold ID and return only the fields in the contract.
- Class name is trimmed and must contain 1-100 characters. totalSeats must be a JSON integer greater than zero.
- Malformed JSON and invalid/missing fields return 400 using the same error envelope.
- Class existence is checked before attempting a hold. Ownership is checked before revealing a state transition. Keep HTTP status and error code stable.

7.5 Endpoint-specific behavior

POST /classes: no auth in the assignment. Return the exact three example fields. The UI refreshes GET /classes after creation. GET /classes returns an empty array rather than null. GET /me/holds returns all the student's holds, newest first. For GET /holds/{id} and GET /me/holds, derive displayed status as EXPIRED when stored status is HELD but expiresAt <= statement_timestamp(); this avoids exposing stale HELD while the sweep worker is catching up.

Optional operational route: GET /health returns 200 only after a quick database ping. It is for Compose health diagnostics and does not replace any required route.

8. Responsive React UI

8.1 Minimal page structure

```
Application shell
  Header: School Admission | Parent | Admin
  Parent view (default)
    Student birth-registration number field
    Class & Shift cards / table
      total seats | held | confirmed | available | Apply
    My Applications
      class | status | expires in | Pay Admission Fee | Cancel
  Admin view
    Create class/shift form
      name | totalSeats | Create
    Inline result / validation message
```

8.2 Responsive behavior

Viewport	Layout
Mobile (320-767 px)	Single-column cards; full-width inputs/buttons; counts wrap; status/actions stack; no horizontal page scrolling.
Tablet (768-1023 px)	Two-column class cards where space permits; My Applications remains readable.
Desktop (1024 px and up)	Centered max-width content; class cards in a responsive grid or compact table; form and summary can sit side-by-side.

Use fluid widths, CSS Grid/Flexbox, `minmax()`, and relative spacing. Set a sensible max content width. Test at 360 px, 768 px and 1280 px. Avoid fixed pixel widths for cards and tables.

8.3 Parent flow

1. Parent enters the birth-registration number and selects “Continue”. Save the valid value in localStorage so the browser can restore My Applications after refresh/reopen. Never log the value.
2. Load GET /classes. Each card is one class/shift pool and shows held, confirmed and available seats.
3. On Apply, disable only that card’s Apply button while the POST is pending. Submit X-Student-Id. Do not display a hold until the server returns 201.
4. On 201, add the returned hold to My Applications and display expiresAt as a countdown. Use the server timestamp; the countdown is informative, not authoritative.
5. On “Pay Admission Fee”, call POST /holds/{id}/confirm. Helper text must say this is a demo simulation and no money is charged.
6. On expiry, show EXPIRED and disable the fee button without reload. The periodic GET /me/holds refresh reconciles the UI with server state.
7. Provide “Forget student ID” to clear the stored value and local app state.

8.4 Admin flow

1. Admin opens the Admin tab in the same SPA.
2. Enter a descriptive pool name such as “Class 6 - Morning Shift” and a positive integer seat count.
3. Submit to POST /classes. On 201, show success and refresh GET /classes.
4. Keep form errors adjacent to the relevant field; do not clear valid inputs after a failed request.

8.5 Polling, countdown and browser recovery

- Refresh GET /classes every 4-5 seconds. Refresh GET /me/holds at the same interval when a valid student ID is present.
- Pause or slow polling when the browser tab is hidden; refresh immediately when it becomes visible.
- Tick visible countdowns every second. At zero, mark that HELD item EXPIRED locally and disable confirmation; the next poll confirms server state.
- Use the server expiresAt value. If using the HTTP Date header to estimate clock offset, expose it through CORS; regardless, server-side expiry remains final.
- If a POST response is lost, do not blindly submit again. Fetch GET /me/holds first; the earlier request may have committed.
- Use bounded retries for GET only. Never automatically retry a payment-confirmation POST without checking state.

8.6 Accessibility and error-free states

- Every input has a visible label; errors are associated with fields using aria-describedby.
- Buttons work with keyboard; focus indicators are visible; status is not communicated by color alone.
- Use aria-live=polite for success/error/expiry updates. Preserve focus after submit.
- Implement loading, empty, error, success and offline/server-unavailable states for every main panel.
- Add a top-level React error boundary with a friendly fallback and retry/reload action. Log technical details only to console in development.

8.7 Suggested visual language

Use a restrained neutral background, one primary accent, clear status text and compact cards. “Available” is the most prominent seat count. Do not add animations, illustrations, complex dashboards or dark-mode work unless time remains after all acceptance tests.

9. Coding structure and conventions

9.1 Repository layout

```
school-admission-system/
  backend/
    Admission.Api/
      Features/
        Classes/       (create, list, get)
        Holds/         (create, get, confirm, cancel, my holds)
      Data/            (DbContext, mappings, migrations)
      Infrastructure/  (clock, id generator, expiry worker)
      Common/          (errors, validation, middleware)
      Program.cs
    Admission.Tests/
  frontend/
    src/
      app/             (App, ErrorBoundary)
      features/
        classes/       (class list, class form)
        holds/         (my holds, countdown, status)
      shared/
        api/           (typed fetch client, contracts)
        components/    (button, input, alert, spinner)
      styles/
  scripts/
    concurrency-test.ps1
  docker-compose.yml
  .env.example
  README.md
  ASSUMPTIONS.md
  AI_USAGE.md
```

9.2 Backend conventions

- Use feature-oriented services: ClassService and HoldService. Keep endpoint lambdas thin.
- Use request/response DTOs; do not return EF entities directly.
- Use async database calls and pass CancellationToken through the call chain.
- Use parameterized SQL for FOR UPDATE, conditional UPDATE and batch expiry only. Normal reads/writes use EF Core.
- One centralized exception/error mapper returns the standard envelope and logs traceId.
- Do not log student IDs, raw request headers or database credentials.
- Use database migrations checked into Git. No EnsureCreated in production/demo startup.

9.3 Frontend conventions

- Define TypeScript interfaces matching API response casing exactly. Do not rename fields at the boundary.
- Use one small API client that adds X-Student-Id only to the four required routes.
- Keep form input state local; keep server data in feature hooks. Avoid a global state library for this scope.
- Clear intervals and event listeners in React effect cleanup. Prevent duplicate submissions with pending state.
- Do not trust locally calculated availability to enable a successful application; API 409 is expected under races.

9.4 IDs and JSON contracts

```
type AdmissionClass = {
  id: string;
  name: string;
  totalSeats: number;
  heldSeats: number;
  confirmedSeats: number;
  availableSeats: number;
```

```
};

type HoldStatus = 'HELD' | 'CONFIRMED' | 'EXPIRED' | 'CANCELLED';

type StudentHold = {
  holdId: string;
  classId: string;
  className: string;
  status: HoldStatus;
  expiresAt: string;
};
```

10. Docker, configuration and startup

10.1 Required environment variables

Variable	Used by	Default / example
POSTGRES_DB	PostgreSQL	admission
POSTGRES_USER	PostgreSQL	admission_app
POSTGRES_PASSWORD	PostgreSQL	Local-only secret from .env; never commit a real secret.
ConnectionStrings__Default	API	Host=db;Port=5432;Database=admission;Username=admission_app;Password=...
HOLD_DURATION_SECONDS	API	600
API_BASE_URL	Frontend runtime config	http://localhost:8080
FRONTEND_ORIGIN	API CORS allow-list	http://localhost:3000
ASPNETCORE_URLS	API listen address	http://+:8080 (second instance: http://+:8081)

10.2 Compose services

Service	Purpose	Ports / profile
db	PostgreSQL with named persistent volume.	Internal 5432; health check with pg_isready.
migrate	One-shot container runs EF migrations and exits.	No host port. API waits for success.
api	Required backend process.	Host 8080 -> container 8080.
web	Nginx serves React build and runtime env.js.	Host 3000 -> container 80.
api-2	Second independent backend process, same DB.	Optional `bonus` profile; host 8081 -> container 8081.

10.3 Compose behavior

- Root docker-compose.yml builds backend and frontend images; users do not install .NET, Node or PostgreSQL on the host.
- The database has a named volume. `docker compose down` keeps data; `docker compose down -v` intentionally deletes it.
- The migration service runs once before API startup. This avoids two API instances racing to apply migrations in the bonus profile.
- The frontend receives API_BASE_URL at container startup through a small generated env.js file. This avoids a rebuild just to change the API URL.

- Configure CORS for FRONTEND_ORIGIN. If frontend/API hostnames change, update both the runtime API URL and the API allow-list.

10.4 Minimal compose sketch

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-admission}
      POSTGRES_USER: ${POSTGRES_USER:-admission_app}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 20

  migrate:
    build: ./backend
    command: ["--migrate-only"]
    env_file: .env
    depends_on:
      db: { condition: service_healthy }
    restart: "no"

  api:
    build: ./backend
    env_file: .env
    environment:
      ASPNETCORE_URLS: http://+:8080
    ports: ["8080:8080"]
    depends_on:
      migrate: { condition: service_completed_successfully }

  web:
    build: ./frontend
    environment:
      API_BASE_URL: ${API_BASE_URL:-http://localhost:8080}
    ports: ["3000:80"]

  api-2:
    profiles: ["bonus"]
    build: ./backend
    env_file: .env
    environment:
      ASPNETCORE_URLS: http://+:8081
    ports: ["8081:8081"]
    depends_on:
      migrate: { condition: service_completed_successfully }

volumes:
  pgdata:
```

The snippet shows the service relationships; add the actual Dockerfiles, restart policies and exact environment values for your app. The `.env` file below is required by this sketch and must not be committed. Configure CORS for FRONTEND_ORIGIN in the API. Validate the committed file with `docker compose config` and a clean `docker compose up --build`.

Example .env.example

```
POSTGRES_DB=admission
POSTGRES_USER=admission_app
POSTGRES_PASSWORD=change-this-local-password
ConnectionStrings__Default=Host=db;Port=5432;Database=admission;Username=admission_app;Password=change-this-local-
password
HOLD_DURATION_SECONDS=600
API_BASE_URL=http://localhost:8080
FRONTEND_ORIGIN=http://localhost:3000
```

Copy `.env.example` to `.env`, then set the same database name, username and password in POSTGRES_* and ConnectionStrings__Default. Use a local-only password; never publish `.env`.

10.5 One-shot migration entry point

The API project should recognize the migration-only argument before starting Kestrel. This makes the Compose `migrate` service run the checked-in EF migrations once, then exit successfully; both API instances wait for it.

```
var migrateOnly = args.Contains("--migrate-only", StringComparer.Ordinal);
var appArgs = args.Where(a => a != "--migrate-only").ToArray();
var builder = WebApplication.CreateBuilder(appArgs);
// Register DbContext and services, then build the app.
var app = builder.Build();

if (migrateOnly)
{
    await using var scope = app.Services.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<AdmissionDbContext>();
    await db.Database.MigrateAsync();
    return;
}

// Map endpoints and start the normal API only in the non-migration path.
// Map the normal API endpoints here.
await app.RunAsync();
```

10.5 Expected commands

```
# First run / normal run
docker compose up --build
# UI:  http://localhost:3000
# API: http://localhost:8080

# Stop containers, keep database data
docker compose down

# Verify data persisted
docker compose up -d

# Optional two-process bonus
docker compose --profile bonus up --build
# API A: http://localhost:8080
# API B: http://localhost:8081

# Intentional full reset (deletes DB volume)
docker compose down -v
```

11. Automated testing and verification

TESTING RULE

Seat safety must be tested against real PostgreSQL. An in-memory provider or SQLite cannot prove PostgreSQL row-lock and partial-index behavior.

11.1 Minimum backend suite

Test	Setup and action	Assertions
Class creation + list	Create class through API, then GET /classes.	201; exact name/totalSeats; counts 0/0/total; array JSON.
One active application	Same student applies twice to same class.	First 201; second 409 ALREADY_APPLIED; only one active row.
Ownership and expiry	Student A creates hold; B confirms; then A confirms after expiry.	B gets 403; expired hold gets 409; no confirmation is written.
Idempotent confirm	Owner confirms the same hold twice.	Both calls return 200 CONFIRMED; one confirmed row; count remains one.

Test	Setup and action	Assertions
Cancellation / release	Create then cancel an unexpired hold.	204; status CANCELLED; next student can obtain released seat.
Automatic expiry	Set test HOLD_DURATION_SECONDS=1-2; wait without calling an API.	GET /me/holds reports EXPIRED and the next applicant obtains the seat.
Concurrency (mandatory)	One class with 20 seats; launch 250 unique-student apply requests together.	At most 20 return 201; remaining requests return 409; DB and GET /classes never exceed 20.
Persistence smoke	Create a class/hold; restart API/Compose without deleting volume.	Rows remain and GET routes return the same committed state.

11.2 Concurrency test protocol

1. Use a real PostgreSQL test database and create one class with a small known capacity (for example 20 seats).
2. Generate 250 distinct valid 17-digit student IDs. Use a start gate/barrier so the HTTP requests are launched together, not sequentially.
3. Call POST /classes/{id}/holds for all students using Task.WhenAll or an equivalent concurrent runner.
4. Assert successful 201 responses are never greater than totalSeats. Expect the remainder to be 409 NO_SEATS_AVAILABLE; unexpected 5xx is a test failure.
5. Read GET /classes/{id} and query PostgreSQL directly in the test fixture. Assert heldSeats + confirmedSeats <= totalSeats.
6. Repeat the test several times with fresh class IDs. Save the command and result summary in README.md.

11.3 Multi-process bonus test

Start Compose with the `bonus` profile so two separate backend containers listen on 8080 and 8081 and share the same database. Extend the concurrency runner to alternate requests between both base URLs (for example, 125 to each). Assert exactly the same capacity invariant. This is the actual multi-process proof; one process bound to two ports is not sufficient.

11.4 Frontend automated tests (bonus)

- Applying disables the selected Apply button while pending and shows a readable 409 NO_SEATS_AVAILABLE message.
- When the countdown reaches zero, the hold is displayed as EXPIRED and Pay Admission Fee is disabled without a page reload.
- After remount/reload, the saved student ID is restored and GET /me/holds repopulates My Applications.
- Admin form rejects blank names and non-positive/non-integer seat counts before calling the API.
- A server/network error renders an alert and leaves the page usable; no unhandled promise rejection occurs.

11.5 Manual responsive smoke test

Viewport / condition	Pass criteria
360 x 800	No horizontal overflow; form and actions fit; cards stack.
768 x 1024	Layout adapts cleanly; counts and buttons remain readable.
1280 x 800	Content width is controlled; no oversized whitespace or stretched fields.
Network offline / API stopped	Friendly error; no blank page; entered student ID remains.
Refresh/reopen	Student ID and current applications are restored.
Expiry while page open	Countdown reaches zero; status changes and confirmation is disabled.

12. Delivery plan (8-11 November)

The assignment deadline is 11 November 2026 at 11:59:59 PM. Finish the implementation and push a working version well before the last hour; leave time for a clean-clone test.

When	Build order	Exit condition
8 Nov (late)	Create repository and solution; add Compose + PostgreSQL + migrations; add class endpoints and DTOs.	Clean docker compose starts DB and API; POST/GET classes work.
9 Nov	Implement hold transaction, unique partial index, confirm/cancel, expiry predicate and sweeper; write backend tests.	Required business rules pass against PostgreSQL; concurrency test passes.
10 Nov	Build parent/admin React views; typed API client; countdown/polling/recovery; frontend tests.	All parent/admin actions work from UI at mobile and desktop widths.
11 Nov (morning)	Wire Docker frontend, env.js runtime API URL, README and assumptions; run default Compose.	Fresh clone starts with only Docker; persistence and API contract verified.
11 Nov (afternoon, buffer)	Run optional two-process bonus; inspect Git history; final public repo review and submit.	No unfinished TODO, no uncommitted files, submission link ready before deadline.

12.1 Build order - do not code everything at once

1. Create and commit the skeleton: backend, frontend, Dockerfiles, compose, README placeholders.
2. Implement database schema/migrations and class read/create endpoints.
3. Implement the create-hold transaction and prove the capacity invariant before building the UI.
4. Implement GET hold / GET my holds / confirm / cancel / expiry cleanup.
5. Build parent UI, then admin form, then responsive polish and error states.
6. Add Docker runtime config, persistence smoke, automated tests and final documents.
7. Only after the mandatory app passes, test the second API process and frontend bonus suite.

12.2 Git commit strategy

Commit in small working increments. Suggested history: `chore: scaffold solution and compose`; `feat(api): add class endpoints`; `feat(db): add holds and indexes`; `feat(admission): implement transactional seat holds`; `feat(admission): add expiry and confirmation`; `feat(ui): add parent admission flow`; `feat(ui): add admin class form`; `test: add postgres concurrency coverage`; `test(ui): add frontend interaction tests`; `docs: finalize README and assumptions`. Avoid one huge final commit.

13. Required repository documents

13.1 README.md

- Project overview, stack and demo limitations (no real authentication/payment).
- Prerequisite: Docker + Docker Compose only for running the application.
- Copy `.env.example` to `.env`; explain which values are local-only.
- Exact `docker compose up --build` command and frontend/API URLs.
- How to stop/restart while preserving data; explain that `down -v` deletes it.
- How to run backend tests, frontend tests and the concurrency test.
- How to run the optional two-process bonus profile and test both ports.
- Screenshots or a short UI walkthrough are optional; do not spend time on decorative assets.

13.2 ASSUMPTIONS.md

Document why each underspecified behavior was chosen. Appendix A contains a copy-ready version aligned with this blueprint. Do not leave assumptions only in code comments.

13.3 AI_USAGE.md

The assignment allows AI tools and asks you to disclose tools, where they were used, mistakes discovered, and how you verified the output. Appendix B provides a fill-in template. Be accurate: record actual prompts/work areas and actual tests you ran; do not claim that AI-generated code was verified if it was not.

13.4 Public GitHub submission

- Create one public repository under your GitHub account and keep backend/ and frontend/ inside it.
- Push the complete commit history, not just a ZIP or one final squashed commit.
- Ensure root docker-compose.yml, Dockerfiles, README.md, ASSUMPTIONS.md and AI_USAGE.md are present.
- Do not upload or publicly share the confidential assignment PDF.
- Submit the repository URL using the submission link stated in the assignment PDF. Do not invent or substitute a different URL.

14. Security, reliability and performance boundaries

14.1 Required for a clean take-home implementation

- Parameterize all SQL. Never concatenate student IDs, hold IDs or class IDs into SQL text.
- Validate input on both client and server. The server is authoritative.
- Do not log birth-registration numbers, passwords, connection strings or request headers containing identifiers.
- Return generic 500/503 messages to the browser; include a traceId for debugging without exposing stack traces.
- Set request cancellation tokens and sensible database command timeouts. Keep seat transactions short.
- CORS allow only the configured frontend origin, not `*` with credentials.
- Use a non-superuser PostgreSQL application role for the API where practical; do not expose database port to the public internet.
- Do not claim that an unauthed birth-registration value proves ownership. This demo is not suitable for real admissions without identity controls.

14.2 What not to optimize prematurely

Do not claim a fixed request-per-second rate without measuring the actual Compose stack. Do not add cache, Redis, a message broker or counter table before profiling. For the take-home, the class-row lock serializes successful creates for one pool while different classes can proceed independently. Run the required concurrency test and report observed results honestly.

14.3 Production-only follow-ups

Before any real deployment: add parent authentication/verified identity, authenticated admin roles, a real payment provider with server-to-server webhook verification and idempotency keys, audit trail, privacy/retention rules for birth-registration data, rate limiting/abuse controls, HTTPS, backups/restore test, monitoring and operational alerts. These are intentionally outside the assignment implementation.

15. Design self-review and corrections

This rewrite was checked against the assignment and the two supplied design drafts. The following potentially misleading or unnecessary elements were deliberately corrected or excluded.

Issue found in earlier draft / common trap	Final decision in this blueprint
One API process listening on two ports presented as the two-process bonus.	Not accepted. Bonus requires two independent API containers/processes, separate host ports, same PostgreSQL database.
64-bit hold ID used while GET /holds/{id} has no owner header.	Use 128-bit cryptographically random IDs. Return only the published response fields.
Unmeasured throughput figures such as 300-1,000 successful requests/sec.	Removed. Throughput must be measured on the actual stack; no invented benchmark claims.
Every operation takes a class-row lock.	Only create-hold takes it. Confirm keeps occupancy constant; cancel/expiry reduce it.
Browser clock or background timer decides capacity.	PostgreSQL time predicates decide expiry/capacity. Browser countdown is visual; worker only materializes EXPIRED.
Stored held/confirmed counters.	Removed. Counts are derived from holds in one aggregate SQL query.
Unique class names or undocumented maximum seats.	Removed. Only positive totalSeats and bounded class-name length are enforced.

Issue found in earlier draft / common trap	Final decision in this blueprint
Treating X-Student-Id as authentication.	Explicitly labelled as demo-only identification.
Real payment integration suggested by the Pay button.	No gateway. The button simulates successful fee payment as the brief instructs.
Required API response fields renamed or removed.	All eight methods/paths/required example fields remain intact.
Frontend relies on POST success response only after network error.	On ambiguous network failure, re-fetch GET /me/holds before another write.
Migration startup race when two API containers boot together.	Use one one-shot migration service; both API processes wait for successful completion.

15.1 Pre-delivery review checklist

- No API path, HTTP method, required header, response field or status has drifted from the assignment.
- No successful concurrent test run produces more active HELD + CONFIRMED holds than totalSeats.
- The student can only confirm their own unexpired hold; repeated confirm is idempotent.
- An expired hold returns its seat without any API call; UI status changes without manual reload.
- Refresh/reopen restores the student's application; class counts refresh automatically.
- Parent and admin workflows are both possible entirely from the UI.
- Default `docker compose up --build` works from a clean clone and preserves data across down/up.
- The optional bonus uses two independent API processes on different ports with one DB.
- Backend and frontend automated tests are green; README commands are current.
- GitHub repository is public, commit history is meaningful, and confidential PDF is absent.
- ASSUMPTIONS.md and AI_USAGE.md match what was actually implemented and tested.

Appendix A. Copy-ready ASSUMPTIONS.md

Copy the following into the repository root as ASSUMPTIONS.md, then edit it to match the actual code. These decisions fill gaps without expanding the assignment scope.

```
# Assumptions

1. A class record represents one grade + shift combination. Each combination
   has an independent seat capacity.
2. The one-active-application rule is per student per class/shift. A student
   may apply to a different class/shift while another application exists.
3. Only HELD applications with expires_at later than PostgreSQL time consume
   seats. CONFIRMED applications consume seats permanently in this assignment.
4. EXPIRED and CANCELLED applications are retained as history. A student may
   apply again after either terminal state.
5. A repeated confirm by the owner of an already CONFIRMED hold returns 200
   and does not change occupancy. A different student receives 403.
6. Cancellation applies only to an unexpired HELD application. Confirmed
   admission is not cancelled/refunded because the brief defines no refund flow.
7. Clicking “Pay Admission Fee” simulates a successful payment. No real payment
   gateway, money transfer, webhook or receipt is integrated.
8. No login/authentication is implemented because the brief specifies a
   17-digit student birth-registration number in X-Student-Id and no login.
   This header is demo identification only, not proof of identity.
9. The admin UI is open in this take-home version because no admin login or
   role API is specified. This is not production security.
10. Class names need not be unique; the generated class ID distinguishes pools.
11. GET /holds/{holdId} does not require X-Student-Id because the published
    endpoint contract does not specify that header. Hold IDs are 128-bit
    cryptographically random values and the response contains minimal data.
12. The student ID is stored in browser localStorage to restore applications
    after refresh/reopen as required. This is only acceptable for the demo;
    do not store real national identifiers this way in production.
13. PostgreSQL is the source of truth for expiry time and seat availability.
    A background worker updates stale HELD rows to EXPIRED; capacity queries
    exclude expired rows even if the worker has not run yet.
14. The default API host port is 8080. The optional bonus runs two independent
    API processes on 8080 and 8081 against one PostgreSQL database; each uses
    the same per-class row lock and unique index, so database state coordinates
    holds rather than process-local memory.
15. HOLD_DURATION_SECONDS defaults to 600 and must be a positive integer.
16. All instants are stored as timestamptz and returned as UTC ISO-8601 values.
```

Appendix B. Copy-ready AI_USAGE.md

The assignment permits AI assistance and requests disclosure. Fill this in honestly. Remove any example that does not match the tools and work you actually used.

```
# AI Usage Disclosure

## Tools used
- Tool: [name and version/model, if known]
- Purpose: [planning, explaining a framework, reviewing code, test ideas, etc.]

## Areas where AI assistance was used
- [Specific file/feature or design area]
- [What was asked and what output was used]
- [What was rewritten or rejected]

## Errors or weak suggestions found
- [Describe the issue, why it was wrong, and the correction]
- [Example: a single API process on two ports does not prove the
  two-independent-process bonus]

## Verification performed by the developer
- [Commands run]
- [Automated tests and their results]
- [Manual UI / responsive checks]
- [Docker clean-start and persistence checks]

## Responsibility
AI output was treated as a draft, not as proof of correctness. I reviewed,
modified and tested the submitted implementation. Any unverified area is
listed explicitly here.
```

Appendix C. README command checklist

- Document `cp .env.example .env` followed by `docker compose up --build`; state that only Docker is required on the host.
- List the frontend URL (`http://localhost:3000`) and required API URL (`http://localhost:8080`).
- Explain that `docker compose down` preserves the named database volume; `docker compose down -v` is a destructive reset.
- Document the optional bonus command `docker compose --profile bonus up --build` and the second API URL (`http://localhost:8081`).
- Provide the exact backend, frontend and concurrency test commands only after running them successfully.
- Never commit `.env`, real credentials or the confidential assignment PDF; commit `.env.example`.

References

[S1] ONNOROKOM PROJUKTI LIMITED, “School Admission System - Assistant Software Engineer Take-Home Assignment,” five-page PDF supplied with this task, especially pages 1-5 (requirements, technology, API contract, mandatory rules, bonus and deliverables).

[R1] Microsoft, .NET and .NET Core official support policy.
https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core

Accessed 8 November 2026. The current table lists .NET 8 end of support on 10 November 2026 and .NET 10 LTS end of support on 14 November 2028.

[R2] React, official version documentation. https://react.dev/versions

Accessed 8 November 2026. The docs list React 19.3 as the latest documented version and retain an archive for React 18. Use a currently patched compatible React 19.x release and commit the lockfile.

FINAL SCOPE STATEMENT

The blueprint specifies a minimal implementation and a way to verify it. It does not certify that future code is bug-free. The acceptance evidence is the executed automated test suite, clean Docker startup, persistence check, responsive UI smoke test and recorded concurrency result.